Dokumentacja projektu - Biblioteka Serial

Kamil Skop

5 stycznia 2026

Spis treści

1 Wprowadzenie **3**
1.1 Wstęp . . . 3
1.2 Omówienie głównych funkcjonalności biblioteki . . . 3
1.3 Kompatybilność formatu binarnego . . . 4
1.3.1 Struktury . . . 4
1.3.2 Rekordy . . . 4
1.3.3 Wsparcie dla ewolucji schematu . . . 4

2 Instalacja biblioteki **5**
2.1 Wymagania . . . 5
2.2 Architektura . . . 5
2.3 Integracja z projektem . . . 5

3 Struktura Języka SRL **6**
3.1 Działanie parsera . . . 6
3.1.1 Usuwanie komentarzy . . . 6
3.1.2 Tokenizacja . . . 7
3.1.3 Konstrukcja wzorców dla struktur danych . . . 8
3.1.4 Weryfikacja poprawności uzyskanego obiektu SerialCompiler . . . 8
3.2 Komendy . . . 9
3.2.1 RECORD / STRUCT . . . 9
3.2.2 DEFINE / DEF . . . 9
3.2.3 LET . . . 10
3.2.4 SET . . . 12
3.2.5 NEST . . . 12
3.2.6 BASE . . . 13
3.2.7 ENSURE . . . 14

4 Publiczne API **16**
4.1 Klasy publiczne i interfejsy . . . 16
4.1.1 SerialCompiler . . . 16
4.1.2 SerialCompiler.Compressor . . . 18
4.1.3 SerialCompiler.Record . . . 18
4.2 Wyjątki . . . 29
4.2.1 SerialCompiler.SyntaxException . . . 29
4.2.2 SerialCompiler.OperationException . . . 29

5 Struktura Projektu **30**

6 Testy na poprawność działania **32**

7 Zastosowane algorytmy **33**
7.1 Format binarny struktury 33
7.1.1 Przykład formatu 33
7.2 Algorytm serializacji struktury 34
7.2.1 Opis 34
7.2.2 Kod 35
7.2.3 Schemat blokowy 36
7.3 Algorytm deserializacji struktury 37
7.3.1 Opis 37
7.3.2 Kod 38
7.3.3 Schemat blokowy 39
7.4 Format binarny rekordu 40
7.4.1 Format binarny grupy zmiennych 40
7.4.2 Przykład formatu 41
7.5 O algorytmach serializacji / deserializacji rekordów 42

8 Podsumowanie i wnioski **42**

1 Wprowadzenie

1.1 Wstęp

Autorem projektu jest Kamil Skop, będący obecnie studentem Politechniki Śląskiej. Biblioteka Serial jest lekkim narzędziem, przeznaczonym do serializacji i deserializacji danych w Javie, których struktura definiowana jest za pomocą dedykowanego, zaprojektowanego specjalnie do tego celu języka DSL (Domain Specific Language), nazywanego dalej SRL.

1.2 Omówienie głównych funkcjonalności biblioteki

Biblioteka Serial pozwala na definiowanie struktur danych o stałym rozmiarze w języku SRL, składających się z typów prymitywnych występujących w języku Java. Wspiera ona zagnieżdżanie struktur / rekordów oraz tworzenie hierarchii dziedziczenia. Istnieje wsparcie dla tablic o stałym rozmiarze, w tym dla ciągów znakowych zakodowanych w formacie UTF-16, reprezentowanych przez tablice znakowe, dopełniane na końcu znakami `NULL`. Struktury danych tak zdefiniowane, można modyfikować, serializować do tablicy bajtów oraz deserializować za pomocą interfejsu publicznego biblioteki.

Przykład kodu SRL

```
STRUCT Vector3
    LET double x ENSURE x IS FINITE
    LET double y ENSURE y IS FINITE
    LET double z ENSURE z IS FINITE

RECORD Entity
    LET long id
    NEST Vector3 position

RECORD Player : Entity
    SET position.y = 5.0
    LET char nickname[16] = "Player"
    ENSURE nickname MATCHES "^[a-zA-Z0-9_-]+$"
```

Przykład użycia w Javie:

```
SerialCompiler compiler = new SerialCompiler(srlSourceCode);
Record gracz = compiler.makeRecord("Player");

gracz.setValue("id", 5L);
gracz.setString("nickname", "Wojtek2005");

byte[] bytes = gracz.serialize();
Record deserialized = compiler.makeRecord(bytes);

System.out.println(deserialized.getReport());
```

1.3 Kompatybilność formatu binarnego

Jedną z największych zalet biblioteki Serial jest możliwość zachowania kompatybilności formatu binarnego w przypadku ewolucji schematu struktur danych. Różni się ona w sposób znaczący między strukturami (`STRUCT`), a rekordami (`RECORD`). Przemianowanie struktury na rekord i odwrotnie nie zachowuje kompatybilności formatu binarnego.

1.3.1 Struktury

Ich format binarny jest prosty. Pola są serializowane dokładnie w takiej kolejności, w jakiej zostały zdefiniowane. To sprawia, że możliwości ewolucji schematu są przy nich bardzo ograniczone. Ich zaletą jest jednak mały rozmiar i szybkość serializacji / deserializacji.

1.3.2 Rekordy

Pola są serializowane razem z nagłówkiem, będącym 3-bajtowym hashem ich nazwy. Hash ten został oparty na funkcji CRC32, usuwając z jej wyjścia najbardziej znaczący bajt. Dzięki takiemu zabiegowi, rekordy są znacznie bardziej odporne na zmiany w strukturze danych, co wiąże się jednak z nieco większym ich rozmiarem.

1.3.3 Wsparcie dla ewolucji schematu

Typ zmiany	**Struktura**	**Rekord**
Zmiana kolejności pól	NIE	TAK
Dodanie pola	Tylko na końcu	TAK
Usunięcie pola	NIE	Poprzez komentarz deprecjonujący
Zmiana rozmiaru tablicy	Tylko powiększenie tablicy prymitywów na końcu	TAK
Dodanie dziedziczenia / zagnieżdżenia	Tylko dodanie komendy `BASE` / `NEST` na końcu	TAK
Usunięcie dziedziczenia / zagnieżdżenia	NIE	Poprzez komentarz deprecjonujący
Zmiana typu pola	NIE	NIE

Tabela 1: Zestawienie kompatybilności formatu binarnego dla różnych typów zmian

Uwaga: Jeśli rekord dziedziczy ze struktury lub ją w sobie zagnieżdża, to wszystkie pola rekordu pochodzące ze struktury potraktowane zostaną podczas serializacji jako część rekordu. To samo tyczy się przechodzenia pól rekordu do struktury.

2 Instalacja biblioteki

2.1 Wymagania

Biblioteka została napisana i przetestowana w środowisku IntelliJ IDEA, korzystając z JDK w wersji 21. W związku z tym, zalecane jest uruchamianie jej właśnie na tej (lub nowszej) wersji Javy. Uruchomienie biblioteki na starszej wersji jest możliwe, ale nie wyklucza się, że może być wtedy konieczna modyfikacja kodu źródłowego.

2.2 Architektura

Biblioteka składa się z jednego pliku "SerialCompiler.java", zawierającego w sobie klasę publiczną SerialCompiler. Opcjonalnie można też wykorzystać plik "SerialCompilerTest.java", zawierający testy jednostkowe w środowisku JUnit5, sprawdzające poprawność działania biblioteki. Decyzja o jednoplikowej architekturze podjęta została ze względu na wymagania, dotyczące przesyłania projektu, gdzie niedozwolonym było wykorzystanie archiwów typu .zip czy .rar, a jedynym akceptowalnym formatem były pliki .txt, zawierające kod źródłowy programu. Decyzja ta została podjęta z pełną świadomością faktu, że tworzenie całego kodu projektu w jednym pliku jest złą praktyką w języku Java.

2.3 Integracja z projektem

Aby zaimportować bibliotekę, wystarczy przenieść plik z jej kodem źródłowym ("SerialCompiler.java") do własnego projektu, z zachowaniem właściwej struktury katalogów dla paczki (ang. package) page.se3.serial.

3 Struktura Języka SRL

3.1 Działanie parsera

Parser przetwarza kod źródłowy języka SRL w kolejnych krokach, opisanych poniżej.

3.1.1 Usuwanie komentarzy

Przed rozpoczęciem parsowania, program usuwa z podanego kodu SRL wszystkie komentarze. Istnieją 3 typy komentarzy:

- **Komentarze jednoliniowe:** Rozpoczynają się od sekwencji `//`. Parser ignoruje wszystkie znaki od tego momentu aż do wystąpienia znaku nowej linii.
- **Komentarze wieloliniowe (blokowe):** Ograniczone znacznikami `/*` (początek) oraz `*/` (koniec). Wszystko, co znajduje się między nimi, jest pomijane, niezależnie od liczby linii.
- **Komentarze deprecjonujące:** Specjalny typ komentarza, służący do oznaczenia pól usuniętych z rekordów. Każda linia, zaczynająca się znakiem _ zostanie uznana za linię przestarzałą. Parser przed kompilacją właściwą zawsze dokonuje kompilacji testowej, w której nie usuwa linii przestarzałych, a jedynie usuwa z nich wiodące znaki _. Kompilacja testowa musi się powieść, inaczej główna zakończy się błędem. Podczas kompilacji głównej linie przestarzałe są całkowicie ignorowane. System ten ma na celu wyeliminowanie ryzyka kolizji hashów pól w przypadku próby dodania nowego pola z hashem już kiedyś istniejącym w rekordzie / strukturze, ale później usuniętym.

Przykład kodu z komentarzami:

```
/*
To jest długi
komentarz
*/

// To jest krótki komentarz

RECORD Pracownik
    LET int id
    LET char imie[32]
    LET char nazwisko[32]
    _LET float stare_pole_1
    __LET float stare_pole_2
```

3.1.2 Tokenizacja

W kroku tokenizacji, każda linia kodu źródłowego SRL przetwarzana jest na zbiór uporządkowany tokenów. Białe znaki traktowane są jako separatory, a puste tokeny pomijane. Poza tym, użycie cudzysłowów np. `"mój tekst"`, `'mój tekst'` komunikuje parserowi, że cały tekst wewnątrz, wraz z cudzysłowami, ma być traktowany jako jeden token. Parser wspiera uciekanie cudzysłowów za pomocą znaku \ np. `"mój tekst \"cytat\""`, przy czym sam znak ucieczki nie jest na tym etapie parsowania usuwany.

Przykład

```
LET char tytul[64] = "Adam Mickiewicz - \"Pan Tadeusz\""
```

Wynik tokenizacji

Proces tokenizacji analizowanej linii skutkuje wygenerowaniem następującej sekwencji tokenów:

Indeks	Wartość tokena
0	`LET`
1	`char`
2	`tytul[64]`
3	`=`
4	`"Adam Mickiewicz - \"Pan Tadeusz\""`

Parser identyfikuje słowa kluczowe w analizowanej linii: `RECORD`, `STRUCT`, `DEFINE`, `DEF`, `LET`, `SET`, `NEST`, `BASE` oraz `ENSURE`. Wykrycie jednego z wymienionych tokenów stanowi punkt podziału instrukcji na poszczególne komendy.

Przykład

Indeks	Wartość tokena
0	`LET`
1	`char`
2	`tytul[64]`
3	`=`
4	`"Adam Mickiewicz - \"Pan Tadeusz\""`
5	`ENSURE`
6	`tytul`
7	`NOT`
8	`MATCHES`
9	`"Juliusz Słowacki"`

Wynik podziału na komendy

```
LET char tytul[64] = "Adam Mickiewicz - \"Pan Tadeusz\""
ENSURE tytul NOT MATCHES "Juliusz Słowacki"
```

3.1.3 Konstrukcja wzorców dla struktur danych

Wszystkie komendy uzyskane w poprzednim kroku wywoływane są sekwencyjnie podczas kompilacji, czego skutkiem jest utworzenie specjalnych obiektów `RecordInfo`, reprezentujących wzorce dla zdefiniowanych w języku SRL struktur danych. Zawierają one spis wszystkich zmiennych, znajdujących się w rekordzie / strukturze oraz ich wartości domyślne. Służą one do kontrukcji obiektów `Record` określonego typu.

3.1.4 Weryfikacja poprawności uzyskanego obiektu SerialCompiler

W ostatnim kroku, następuje weryfikacja, czy wszystkie uzyskane obiekty `RecordInfo` są prawidłowe. W tym celu, wykonywane są następujące kroki:

1. Stworzenie listy wszystkich zdefiniowanych struktur danych (obiektów `Record`), wypełnionych wartościami domyślnymi.
2. Zweryfikowanie, czy wszystkie obiekty `Record` spełniają założenia, zdefiniowane za pomocą komendy `ENSURE`.
3. Serializacja i deserializacja każdego obiektu `Record`, aby upewnić się, że przebiega ona prawidłowo.

W przypadku niepowodzenia któregokolwiek z powyższych punktów, proces kompilacji zostaje przerwany z błędem.

3.2 Komendy

Poniżej, przedstawione zostały wszystkie komendy, występujące w języku SRL.

3.2.1 RECORD / STRUCT

Komendy `RECORD` i `STRUCT` służą do zakomunikowania parserowi, że rozpoczyna się definicja nowego rekordu / struktury, w zależności od wybranego słowa kluczowego. Opcjonalnie, można umieścić w nich informację o dziedziczeniu, bez konieczności użycia komendy `BASE`.

Składnia

```
STRUCT/RECORD nazwa
STRUCT/RECORD nazwa : baza_1 baza_2 .. baza_n
```

Przykład

```
STRUCT Baza1
    LET int a = 1

RECORD Baza2
    LET int b = 2

RECORD Pochodny : Baza1 Baza2
    LET int c = 3
```

Rekord `Pochodny` dziedziczy wszystkie pola wraz z wartościami domyślnymi ze struktury `Baza1` oraz z rekordu `Baza2`, a następnie dodaje do nich własne pola.

3.2.2 DEFINE / DEF

Komendy `DEFINE` i `DEF` działają jak makra tekstowe, zastępując wskazane tokeny bezpośrednio przed ich interpretacją. Różnią się one przede wszystkim zasięgiem: `DEFINE` ma charakter globalny, natomiast `DEF` jest makrem lokalnym, którego ważność wygasa wraz z zamknięciem bieżącego rekordu lub struktury. W przypadku wystąpienia nazw o tym samym brzmieniu, makro lokalne `DEF` ma zawsze priorytet nad globalnym `DEFINE`. Należy zaznaczyć, że same instrukcje definicji nie podlegają mechanizmowi zastępowania tokenów.

Składnia

```
DEFINE/DEF szukaj zamien
DEFINE/DEF szukaj = zamien
```

Przykład

```
// zaleca się używanie znaku '$' przed stałymi
DEFINE $global-value = 100

RECORD Rekord1
    DEF $local-value = 50
    LET int ar1 = $local-value

RECORD Rekord2
    // tutaj $local-value już nie istnieje
    LET int ar2 = $global-value
```

3.2.3 LET

Komenda `LET` służy do definiowania zmiennych oraz tablic wewnątrz rekordu / struktury oraz do przypisywania im wartości domyślnych. Jeśli wartość domyślna nie zostanie sprecyzowana, zostanie ona wypełniona zerami.

Definiowanie zmiennych: Zmienne w języku SRL mogą przyjąć postać wszystkich 8 typów prymitywnych, które znajdziemy w języku Java. Do ich parsowania zostały wykorzystane wbudowane funkcje parsujące takie, jak `Short.parseShort(...)`. Wyjątek stanowią typy `boolean` oraz `char`, które otrzymały dedykowane dla nich parsery.

Składnia

```
LET typ_prymitywny nazwa
LET typ_prymitywny nazwa = wartość_domyslna
```

Przykład

```
RECORD Rekord
    LET byte bitowa = -5
    LET short krotka = -10
    LET int liczba = 15
    LET long dluga = 20
    LET float ulamek1 = 3.5
    LET double ulamek2 = 3.3552
    LET char znak = 'A' // można też użyć %65, '\u0041' (Unicode)
       albo samotnej litery A
    LET boolean logiczna = true // można też użyć 0 albo 1; wyraż
       enia false, true są case-insensitive
```

Definiowanie tablic: Tablice są w rzeczywistości powtórzonymi `N` razy deklaracjami danej zmiennej, gdzie `N` to wielkość tablicy. Pisząc: `LET int array[3]` deklarujemy tak naprawdę zmienne: `array[0]`, `array[1]` oraz `array[2]`. Mimo to, zaleca się używanie tablic zamiast zmiennych z suffixem, ponieważ pozwala to serializatorowi na pewne optymalizacje w wielkości formatu binarnego, a także jest to wygodniejsze dla człowieka.

Składnia

```
LET typ_prymitywny nazwa[wielkość]
LET typ_prymitywny nazwa[wielkość] = w1 w2 .. wn
LET typ_prymitywny nazwa[wielkość] = w1 w2 .. wn ...
```

Przykład

```
RECORD Rekord
    // Pusta tablica, wypełniona zerami
    LET int tablicaA[5]

    // Tablica, zawierająca kolejne elementy od 1 do 3, a reszta
       wypełniona zerami
    LET int tablicaB[5] = 1 2 3

    // Tablica, zawierająca kolejne elementy od 1 do 3, a reszta
       wypełniona elementem ostatnim, czyli 3.
    LET int tablicaC[5] = 1 2 3 ...

    // Tablica znaków, tożsama z ciągiem znaków "ABCD".
    LET char ciagZnakow[16] = A B C D
```

Definiowanie łańcuchów znakowych: Tablice typu `char` mogą być w zależności od kontekstu interpretowane przez bibliotekę również jako łańcuchy znakowe. W związku z tym, do parsera dodana została składnia, pozwalająca na określenie wartości domyślnej łańcucha znakowego w przyjazny wizualnie sposób.

Składnia

```
LET char nazwa[wielkość] = "Jakiś ciąg znaków\n"
```

Przykład

```
RECORD Rekord
    // Poniższe deklaracje są ze sobą tożsame.
    LET char text1[16] = "Ala ma kota\n"
    LET char text2[16] = A L A ' ' M A ' ' K O T A '\n'
```

3.2.4 SET

Komenda `SET` pozwala nadpisywać zdefinione już wcześniej wartości domyślne zmiennych oraz tablic. Jest to szczególnie przydatne przy zagnieżdżaniu i dziedziczeniu rekordów, o których więcej wspomniane zostało w dalszych punktach dokumentacji.

Składnia

```
SET nazwa_pola
SET nazwa_pola = wartość_domyślna
```

Przykład

```
RECORD Baza
    LET int varA = 5
    LET int varB = 5
    LET char text[16] = "ab cd"

RECORD Pochodna : Baza
    SET varA // zostanie ustawione 0
    SET varB = 3
    SET text = a b // nowy łańcuch: "ab"
```

3.2.5 NEST

Komenda `NEST` pozwala na zagnieżdżanie rekordów. Działa to na zasadzie kopiowania wszystkich zmiennych oraz założeń zdefiniowanych za pomocą komendy `ENSURE` do rekordu nadrzędnego, poprzedzając każdą nazwę zmiennej nazwą zagnieżdżenia, z kropką jako separatorem. Zmienne wklejone do rekordu za pomocą komendy `NEST` można dowolnie modyfikować komendą `SET`, odwołując się do nich przez nazwę zagnieżdżenia i operator kropki.

Składnia

```
NEST nazwa_rekordu nazwa_gniazda
NEST nazwa_rekordu nazwa_tablicy_gniazd[wielkość]
```

Przykład

```
RECORD Baza
    LET int var = 5
    LET char text[16] = a b c d

RECORD Pochodna
    NEST Baza gniazdo
        SET gniazdo.var = 3
        SET gniazdo.text = "abc"
    // możliwe są tablice zagnieżdżeń
    NEST Baza gniazda[3]
        SET gniazda[1].var = 99
```

3.2.6 BASE

Komenda `BASE` pozwala na dziedziczenie rekordów. Działa to na zasadzie kopiowania wszystkich zmiennych oraz założeń zdefiniowanych za pomocą komendy `ENSURE` do rekordu nadrzędnego. Zmienne wklejone do rekordu za pomocą komendy `BASE` można dowolnie modyfikować komendą `SET`. Alternatywnie, zamiast komendy `BASE` można użyć dziedziczenia wewnątrz komendy `RECORD` / `STRUCT`.

Składnia

```
BASE nazwa_rekordu
```

Przykład

```
RECORD Baza
    LET int var = 5
    LET char text[16] = a b c

RECORD Pochodna
    BASE Baza
    SET var = 3
    SET text = "abc"
```

3.2.7 ENSURE

Komenda `ENSURE` stanowi rozbudowane narzędzie do określania, kiedy struktura / rekord są poprawne. Reguły określić można dla zmiennych, tablic oraz ciągów znaków. Publiczne API udostępnia metodę do weryfikacji integralności rekordu, która w przypadku wykrycia niezgodności, natychmiast go naprawia (przywraca pole do wartości domyślnej). Weryfikacja integralności może również odbyć się niejawnie, podczas deserializacji. Jest jednak możliwe wywołanie metody deserializującej w taki sposób, aby weryfikacja nie zaszła. Domyślne wartości pól muszą spełniać wszystkie reguły `ENSURE`, inaczej kompilacja zakończy się błędem.

Schemat komendy: Warunki w komendzie `ENSURE` zapisuje się w postaci DNF (Disjunctive Normal Form), korzystając z operatorów `OR` oraz `AND` bez użycia nawiasów. Pozwala to na opis niezwykle złożonych zdań logicznych. Każdy z warunków elementarnych musi składać się z co najmniej dwóch tokenów: operatora oraz argumentu. Dopisanie tokenu `NOT` przed warunkiem, neguje go.

```
// Składnia komendy
ENSURE nazwa_pola [warunki_w_DNF]

// Składnia warunku elementarnego
operator argument
NOT operator argument
```

Operatory klasyczne: Są to >, >=, <, <=, == oraz `!=`. Działają one na zmiennych i reprezentują porównania pola z argumentem. Jeśli w miejscu pola zostanie podana tablica, operatory te odnoszą się do każdego jej elementu oddzielnie. Porównania te dla liczb zmiennoprzecinkowych działają zgodnie ze standardem IEEE 754.

Przykład

```
RECORD Rekord
    LET double a = -1
    LET double arr[100] = 3 ...
    ENSURE a > 0 AND < 10 OR == -1
    ENSURE arr != 5 // dla każdego elementu tablicy arr
```

Operator IS: Użycie tego operatora ma sens tylko w przypadku liczb zmiennoprzecinkowych (float oraz double). Dla każdej innej wartości, zwraca on `false`. Jest to operator, należący do rodziny klasycznych, co oznacza, że działa on na pojedynczych zmiennych lub zbiorczo na tablicach. Przyjmuje on 3 opcje jako argument: `FINITE`, `INFINITE` oraz `NAN`, które sprawdzają, czy liczba zmiennoprzecinkowa należy do jednej z tych trzech grup, zgodnie ze standardem IEEE 754.

Przykład

```
RECORD Rekord
    LET double a = 5
    LET double b = -Infinity
    LET double c = NaN
    LET double d = 0
    ENSURE a IS FINITE
    ENSURE b IS INFINITE
    ENSURE c IS NAN
    ENSURE d NOT IS NAN
```

Operatory łańcuchowe: Są to operatory, które sprawdzają poprawność łańcucha znaków i traktują go jako integralną całość. Oznacza to, że w przypadku błędnej wartości, cały ten łańcuch zostanie przywrócony do wartości domyślnej. Do operatorów łańcuchowych należą `EQUALS`, który sprawdza równość z podanym jako argument łańcuchem, a także `MATCHES`, który sprawdza, czy w badanym łańcuchu występuje dopasowanie wyrażenia regularnego regex, podanego jako argument.

Przykład

```
RECORD Rekord
    LET char imie[64]
    ENSURE imie NOT EQUALS "Adolf"
    ENSURE imie MATCHES "^[\\p{L} -]*$"
```

Kolejność sprawdzania warunków: Warunki sprawdzane są zgodnie z zasadą short-circuit evaluation co oznacza, że gdy wartość wyrażenia staje się pewna, np. po wykazaniu prawdy w jednej z alternatyw, ewaluacja jest natychmiast przerywana.

Moment kompilacji warunków: Warunki nie są kompilowane tak długo, jak nie będą musiały zostać użyte. Może się zdarzyć, że niepoprawny warunek wyrzuci błąd już na etapie kompilacji (`SyntaxException`), jeśli parser będzie chciał go sprawdzić podczas weryfikacji rekordów domyślnych, jednak istnieje również możliwość, że błąd pojawi się dopiero przy sprawdzaniu rekordu rzeczywistego (`OperationException`).

4 Publiczne API

Biblioteka Serial udostępnia zestaw klas oraz metod publicznych, za pomocą których jest możliwe jej użytkowanie. Poniżej przedstawiono dokumentację interfejsu publicznego.

4.1 Klasy publiczne i interfejsy

Poniżej omówiono klasy użytkowe.

4.1.1 SerialCompiler

Jest to klasa, której obiekt reprezentuje sparsowany plik języka SRL. Pełni ona rolę fabryki obiektów klasy zagnieżdżonej `SerialCompiler.Record`.

SerialCompiler(String code, Compressor compressor) `konstruktor`

Opis: Kompiluje kod na podstawie kodu źródłowego SRL i tworzy fabrykę obiektów z określonym algorytmem kompresji danych binarnych.

- **Parametry:**
  - `code` – kod w formacie SRL.
  - `compressor` – obiekt implementujący interfejs `SerialCompiler.Compressor`.
- **Wyjątki:**
  - `SyntaxException` – rzucany, gdy składnia kodu jest niepoprawna.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

SerialCompiler(String code) `konstruktor`

Opis: Przeciążenie konstruktora, wyłączające kompresję danych binarnych.

Tworzenie rekordów

Record makeRecord(String recordType) `metoda`

Opis: Tworzy nową, domyślną instancję obiektu `Record` na podstawie podanego typu rekordu / struktury.

- **Parametry:**
  - `recordType` – nazwa identyfikacyjna rekordu / stuktury zdefiniowana w języku SRL.
- **Zwraca:**
  - obiekt klasy `Record` o podanym typie, wypełniony wartościami domyślnymi, zdefiniowanymi w języku SRL.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy podany typ rekordu jest nieprawidłowy.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Record makeRecord(byte[] bytes, boolean autoFix = true) `metoda`

Opis: Tworzy nową instancję obiektu `Record` na podstawie danych binarnych, uprzednio przepuszczając je przez algorytm dekompresji, podany w konstruktorze.

- **Parametry:**
  - `bytes` – tablica bajtów, reprezentująca dane binarne do deserializacji.
  - `autoFix` – czy rekord ma zostać naprawiony w przypadku niezgodności w regułach `ENSURE`?
- **Zwraca:**
  - zdeserializowany obiekt klasy `Record` o typie odczytanym z formatu binarnego lub `"__EMPTY_RECORD__"`, jeśli format jest niepoprawny.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy podany typ rekordu jest nieprawidłowy.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

4.1.2 SerialCompiler.Compressor

Jest to interfejs, reprezentujący używany algorytm kompresji / dekompresji przy serializacji / deserializacji.

byte[] compress(byte[] data) `metoda interfejsu`

Opis: Kompresuje przekazane dane binarne zgodnie z zaimplementowanym algorytmem.

- **Parametry:**
  - `data` – tablica bajtów zawierająca surowe dane do skompresowania.
- **Zwraca:**
  - nową tablicę bajtów, zawierającą skompresowane dane.

byte[] decompress(byte[] data) `metoda interfejsu`

Opis: Dekompresuje przekazane dane binarne, przywracając je do pierwotnej postaci przed kompresją.

- **Parametry:**
  - `data` – tablica bajtów zawierająca skompresowane dane.
- **Zwraca:**
  - nową tablicę bajtów, zawierającą oryginalne, rozpakowane dane lub `null` w przypadku problemów np. wykrycia bomby dekompresyjnej.

4.1.3 SerialCompiler.Record

Jest to obiekt, reprezentujący strukturę / rekord (w zależności od definicji w SRL), który pełni rolę wrappera na słownik, udostępniającego metody przedstawione poniżej. Zmienne przechowywane są w nim pod kluczami, w sposób spłaszczony np. `zmienna`, `tablica[5]`, `gniazda[2].zmienna`. Klucz ten lub jego prefix nazywamy ścieżką – `path`.

String RECORD_TYPE `pole finalne`

Opis: Przechowuje typ rekordu, zdefiniowany w języku SRL.

Zarządzanie zmiennymi

Record setValue(String path, Object value) `metoda`

Opis: Nadpisuje pojedynczą zmienną w rekordzie podaną wartością.

- **Parametry:**
  - `path` – klucz, którego wartość będzie modyfikowana np. `nested[2].variable`.
  - `value` – nowa wartość zmiennej, rzutowana do klasy `Object`.
- **Zwraca:**
  - bieżący obiekt klasy `Record`.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy ścieżka do zmiennej jest nieprawidłowa lub wartość jest niepoprawnego typu.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

<T> T getValue(String path, Class<T> clazz) `metoda`

Opis: Pobiera pojedynczą zmienną ze wskazanej ścieżki i rzutuje ją na podaną klasę.

- **Parametry:**
  - `path` – klucz, którego wartość będzie pobierana np. `nested[2].variable`.
  - `clazz` – klasa typu docelowego, który ma zostać pobrany np. `Boolean.class`.
- **Zwraca:**
  - wartość znajdującą się pod wskazaną ścieżką, zrzutowaną do typu `T` lub `null`, jeśli wartość nie istnieje lub nie jest identycznego typu z klasą `clazz`.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy typ `clazz` jest prymitywem.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Object getValue(String path) `metoda`

Opis: Pobiera pojedynczą zmienną ze wskazanej ścieżki bez rzutowania.

- **Parametry:**
  - `path` – klucz, którego wartość będzie pobierana np. `nested[2].variable`.
- **Zwraca:**
  - wartość znajdującą się pod wskazaną ścieżką lub `null`, jeśli wartość nie istnieje.
- **Wyjątki:**
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Zarządzanie tablicami

Record setArray(String path, Object[] array) `metoda`

Opis: Wstawia tablicę typów prostych do podanej ścieżki.

- **Parametry:**
  - `path` – ścieżka, pod którą znajduje się tablica np. `array`, gdy wartości kluczy to `array[0]`, `array[1]` itd.
  - `array` – tablica wrapperów dla typu prymitywnego np. `Float[]`, którym ma zostać wypełniona tablica.
- **Zwraca:**
  - bieżący obiekt klasy `Record`.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy wielkości tablic są niezgodne lub wartości są niepoprawnego typu.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

<T> T[] getArray(String path, Class<T> clazz) `metoda`

Opis: Pobiera tablicę typów prostych z podanej ścieżki.

- **Parametry:**
  - `path` – ścieżka, pod którą znajduje się tablica np. `array`, gdy wartości kluczy to `array[0]`, `array[1]` itd.
  - `clazz` – klasa typu docelowego, który ma zostać pobrany np. `Boolean.class` dla tablicy `Boolean[]`.
- **Zwraca:**
  - największą możliwą tablicę podanego typu, którą uda się uzyskać, pobierając wartości z kolejnych kluczy.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy typ `clazz` jest prymitywem.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Zarządzanie ciągami znaków

Record setString(String path, String str) `metoda`

Opis: Umieszcza ciąg znaków wewnątrz tablicy typu `Character` w podanej ścieżce, wypełniając niezagospodarowane miejsce znakiem `NULL`.

- **Parametry:**
  - `path` – ścieżka, pod którą znajduje się tablica typu znakowego np. `text`, gdy wartości kluczy to `text[0]`, `text[1]` itd.
  - `str` – ciąg znaków, który ma zostać wstawiony do tablicy.
- **Zwraca:**
  - bieżący obiekt klasy `Record`.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy ciąg znaków jest zbyt długi, żeby zmieścić się w tablicy.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

String getString(String path, boolean forceUnicode = true) `metoda`

Opis: Przekształca tablicę typu `Character` w podanej ścieżce na ciąg znaków, usuwając z jego końca znaki `NULL`.

- **Parametry:**
  - `path` – ścieżka, pod którą znajduje się tablica typu znakowego np. `text`, gdy wartości kluczy to `text[0]`, `text[1]` itd.
  - `forceUnicode` – czy niepoprawne sekwencje `Unicode` powinny zostać zastąpione znakiem zastępczym o kodzie `hex = FFFD`?
- **Zwraca:**
  - ciąg znaków, uzyskany z podanej ścieżki do tablicy znakowej lub pusty ciąg, jeśli taka tablica nie istnieje.
- **Wyjątki:**
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Zarządzanie rekordami

Record setRecord(String path, Record record) `metoda`

Opis: Wkleja każdą zmienną podanego rekordu do podanej ścieżki w bieżącym rekordzie. Ignoruje zmienne, dla których wklejenie by się nie powiodło.

- **Parametry:**
  - `path` – ścieżka, do której rekord ma zostać wklejony np. `gniazdo`, gdy chcemy wkleić jego zmienne z przedrostkiem `gniazdo.` lub pusty ciąg, aby wkleić je bez przedrostka.
  - `record` – obiekt klasy `Record`, który wklejony zostanie do obiektu bieżącego.
- **Zwraca:**
  - bieżący obiekt klasy `Record`.
- **Wyjątki:**
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Record getRecord(String path, String recordType) `metoda`

Opis: Przepisuje każdą zmienną, zaczynającą się od odpowiedniego przedrostka z bieżącego rekordu do nowego rekordu podanego typu, usuwając ten przedrostek. Ignoruje zmienne, dla których wklejenie by się nie powiodło.

- **Parametry:**
  - `path` – ścieżka, z której rekord ma zostać przepisany np. `gniazdo`, gdy chcemy przepisywać zmienne z przedrostkiem `gniazdo.` lub pusty ciąg, aby przepisać je bez przedrostka.
  - `recordType` – typ rekordu, do którego mają zostać przepisane wartości.
- **Zwraca:**
  - nowy obiekt klasy `Record` typu `recordType`, do którego przepisane zostały zmienne.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy docelowy typ nie istnieje.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Konwersje rekordów

Record castTo(String recordType) `metoda`

Opis: Przepisuje wszystkie zmienne z bieżącego rekordu do nowego obiektu klasy `Record` podanego typu. Ignoruje zmienne, dla których wklejenie by się nie powiodło.

- **Parametry:**
  - `recordType` – typ rekordu, do którego mają zostać przepisane wartości.
- **Zwraca:**
  - nowy obiekt klasy `Record` typu `recordType`, do którego przepisane zostały zmienne.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy docelowy typ nie istnieje.
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Record clone() `metoda`

Opis: Wykonuje kopię głęboką rekordu, a następnie ją zwraca.

Resetowanie zmiennych / pól

Record resetValue(String path) `metoda`

Opis: Przywraca pojedynczą zmienną do wartości domyślnej, zdefiniowanej w kodzie SRL.

- **Parametry:**
  - `path` klucz, którego wartość zostanie zresetowana np. `nested[2].variable`.
- **Zwraca:**
  - bieżący obiekt klasy `Record`.
- **Wyjątki:**
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Record resetArray(String path) `metoda`

Opis: Przywraca tablicę prymitywów do wartości domyślnej, zdefiniowanej w kodzie SRL.

- **Parametry:**
  - `path` – ścieżka, pod którą znajduje się tablica np. `array`, gdy wartości kluczy to `array[0]`, `array[1]` itd.
- **Zwraca:**
  - bieżący obiekt klasy `Record`.
- **Wyjątki:**
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Record resetTree(String path) `metoda`

Opis: Przywraca wartości domyślne dla zmiennej wskazywanej przez `path` oraz dla całej podległej jej hierarchii danych. Resetowany jest klucz główny `path` oraz wszystkie klucze zaczynające się przedrostkami strukturalnymi: `path.` oraz `path[`.

- **Parametry:**
  - `path` – ścieżka do hierarchii danych, która ma zostać zresetowana.
- **Zwraca:**
  - bieżący obiekt klasy `Record`.
- **Wyjątki:**
  - `NullPointerException` – rzucany, gdy którykolwiek parametr ma wartość `null`.

Inne metody

byte[] serialize() `metoda`

Opis: Dokonuje serializacji rekordu / struktury do postaci binarnej, a następnie kompresuje wynik podanym w konstruktorze klasy `SerialCompiler` algorytmem.

- **Zwraca:**
  - nową tablicę bajtów, reprezentującą postać binarną rekordu / struktury.

boolean checkAndRepair() `metoda`

Opis: Sprawdza wszystkie warunki `ENSURE` i jeśli znajdzie niezgodności, przywraca pole, którego dane warnki dotyczą do stanu domyślnego.

- **Zwraca:**
  - wartość logiczną określającą, czy jakakolwiek naprawa została dokonana.
- **Wyjątki:**
  - `OperationException` – rzucany, gdy wystąpi błąd w sprawdzaniu warunków `ENSURE` np. niepoprawny regex.

SerialCompiler getCompiler() `metoda`

Opis: Zwraca obiekt klasy `SerialCompiler`, za pomocą którego utworzony został bieżący rekord.

LinkedHashMap<String, Class<?>> getOrderedVariables() `metoda`

Opis: Zwraca nowy obiekt typu `LinkedHashMap`, mapujący wszystkie ścieżki do typów wartości się w nich znajdujących. Pola uporządkowane są zgodnie z kolejnością deklaracji w kodzie SRL.

String getReport() `metoda`

Opis: Zwraca raport z zawartości rekordu w formacie czytelnym dla człowieka. Pola uporządkowane są zgodnie z kolejnością deklaracji w kodzie SRL.

Metody klasy Object

String toString() `metoda`

Opis: Zwraca tekstową reprezentację obiektu, zawierającą informację o klasie oraz typie rekordu w formacie `"SerialCompiler.Record::TYP_REKORDU"`.

boolean equals(Object other) `metoda`

Opis: Porównuje bieżący rekord z innym obiektem pod kątem równości logicznej.

- **Parametry:**
  - `other` – obiekt, z którym ma zostać porównany bieżący rekord.
- **Zwraca:**
  - `true`, jeśli oba obiekty należą do klasy `Record`, pochodzą z tego samego kompilatora, mają ten sam typ oraz identyczną zawartość wszystkich zmiennych; `false` w przeciwnym razie.

int hashCode() `metoda`

Opis: Generuje kod hash na podstawie referencji do obiektu klasy nadrzędnej `SerialCompiler`, typu rekordu oraz wartości wszystkich zmiennych, z uwzględnieniem ich kolejności deklaracji.

4.2 Wyjątki

Poniżej omówiono zaimplementowane klasy wyjątków. Wszystkie rzucane w bibliotece specjalne wyjątki posiadają wiadomość o błędzie, którą można odczytać, korzystając z metody `getMessage()`.

4.2.1 SerialCompiler.SyntaxException

Jest to wyjątek, dziedziczący po `Exception` rzucany w konstruktorze klasy `SerialCompiler`, gdy wystąpi błąd składni języka SRL lub inny nieoczekiwany błąd.

Integer getLine() `metoda`

Opis: Zwraca linię, na której wystąpił błąd w kodzie SRL lub `null`, gdy nie jest możliwe jej określenie.

4.2.2 SerialCompiler.OperationException

Jest to wyjątek, dziedziczący po `RuntimeException` rzucany, gdy w bibliotece wystąpi nieoczekiwany błąd, niezwiązany z kompilacją kodu SRL. Wyjątek ten nie posiada interfejsu publicznego poza dostępnym w ramach klasy `RuntimeException`.

5 Struktura Projektu

Projekt składa się z dwóch plików:

- `SerialCompiler.java` – plik, zawierający implementację biblioteki wraz z jej interfejsem publicznym.
- `SerialCompilerTest.java` – plik, zawierający testy jednostkowe w środowisku JUnit5, stanowiący uzupełnienie dla implementacji.

Pakiet `page.se3.serial` (plik `SerialCompiler.java`) zawiera komponenty tworzące bibliotekę. Jej fundamentem jest publiczna klasa `SerialCompiler`, wewnątrz której zdefiniowano klasy i interfejsy zagnieżdżone, stanowiące API dla użytkownika końcowego. Poniżej przedstawiono wykaz tych elementów wraz z opisem ich odpowiedzialności.

Interfejsy

- `Globals` – interfejs, zawierający stałe globalne np. słowa kluczowe języka SRL.
- `SerialCompiler.Compressor` – interfejs publiczny, reprezentujący algorytm kompresji.

Klasy statyczne

- `Utils` – klasa, w której znajdują się różne, przydatne w projekcie metody statyczne.
- `Validator` – klasa, walidująca poprawność użytych nazw rekordów i zmiennych przy użyciu wyrażeń regularnych regex.
- `Parser` – klasa, zawierająca w sobie własne implementacje klas, parsujących niektóre prymitywy oraz ciągi znaków.
- `SizeReducer` – klasa, odpowiedzialna za algorytm kompresji stratnej czterobajtowej wielkości tablicy do jednego bajta.
- `Tokenizer` – klasa, dokonująca różnorodnych operacji na grupach w postaci tekstu i tokenach.

Klasy referencyjne

- `DoubleHashMap` – wrapper na dwa obiekty typu `LinkedHashMap`, pozwalający na szybki dostęp przez klucze do wartości oraz przez wartości do kluczy.
- `PrimitiveLike` – klasa, reprezentująca jeden z ośmiu typów prymitywnych, dostępnych w języku Java, udostępniająca metody przydatne przy pracy z prymitywem danego typu, nieznanego na etapie kompilacji.
- `ArrayLabel` – klasa, dzięki której można sprawdzić, czy dany klucz reprezentuje zapis tablicowy (posiada indeks na końcu), a także wytwarzać nowe klucze o wybranym indeksie.

- `HashLabel` – klasa, która pozwala zredukować klucz do zapisu bezindeksowego (np. `path[1].zmienna[5]` przekształcone zostanie do `path[*].zmienna[*]`), a następnie tworzyć klucze, wypełniając gwiazdki odpowiednimi indeksami. To z zapisu bezindeksowego powstaje hash danego pola przy serializacji rekordu, stąd nazwa klasy.
- `GroupIterator` – klasa, pozwalająca iterować po tablicach wielowymiarowych, gdzie licznikiem jest tablica o wielkości liczby wymiarów.
- `Ensuration` – klasa, reprezentująca zbiór warunków, zdefiniowanych za pomocą komendy `ENSURE`.
- `Ensuration.Condition` – klasa, reprezentująca pojedynczy segment warunku, zdefiniowany za pomocą komendy `ENSURE`.
- `VariableInfo` – niemutowalna klasa, reprezentująca cechy danego pola, zdefiniowanego w języku SRL. Może się zdarzyć, że wiele pól dzieli ten sam obiekt, aby zaoszczędzić miejsce w pamięci.
- `RecordInfo` – klasa, reprezentująca cechy danego rekordu / struktury. Interpretuje ona również komendy, działające na rekordach / strukturach, tzn. `LET`, `SET`, `NEST` itp.
- `SerialCompiler` – główna klasa publiczna, reprezentująca skompilowany kod SRL. Posiada metody, umożliwiające tworzenie rekordów.
- `SerialCompiler.Record` – klasa publiczna, reprezentująca instancję rekordu / struktury, utworzoną przez użytkownika.
- `SerialCompiler.Serializer` – klasa, służąca do serializacji i deserializacji struktur / rekordów.

Klasy wyjątków

- `SerialCompiler.SyntaxException` – wyjątek rzucany, gdy wystąpi błąd kompilacji kodu SRL.
- `SerialCompiler.OperationException` – wyjątek rzucany w pozostałych przypadkach.

6 Testy na poprawność działania

Weryfikacja poprawności implementacji biblioteki została przeprowadzona przy użyciu środowiska `JUnit5`, przeznaczonego do tworzenia testów jednostkowych. W pliku `SerialCompilerTest.java` utworzono 96 testów jednostkowych, testujących działanie różnorodnych obszarów działania biblioteki w następujących klasach zagnieżdżonych:

Interpretacja składni języka SRL

- `BasicOperations` – podstawowe komendy np. `LET`, `SET`, `DEFINE` itp.
- `ArrayDeclarations` – sposoby deklaracji tablic np. wylistowanie elementów (`1 2 3 4`), czy testowanie operatora dopełnienia (`...`).
- `StringDeclarations` – sposoby deklaracji ciągów znakowych, w tym interpretacja polskich znaków, emoji i ciągów zawierających i zakończonych znakiem `NULL`.
- `PrimitiveParsingTests` – parsowanie typów prymitywnych za pomocą metod wbudowanych w Javę oraz własnych implementacji.
- `Deprecation` – interpretacja komentarzy deprecjonujących w tym, czy są one poprawnie usuwane i czy wykrywane są kolizje hashów.
- `Ensuring` – działanie komendy `ENSURE` i poprawność weryfikacji warunków oraz przywracania zmiennych / tablic / ciągów znaków do wartości domyślnych.
- `IllegalSyntax` – sprawdzanie, czy wyjątek `SyntaxException` jest rzucany w przypadku różnych typów błędów składniowych.
- `OtherAssumptions` – testy, których nie udało się zakwalifikować do pozostałych kategorii.

Działanie publicznego API

- `ApiTests` – pobieranie i przekształcenia rekordów za pomocą metod interfejsu publicznego, w tym rzutowanie i klonowanie rekordów, resetowanie zmiennych, a także weryfikacja rzucania wyjątku `NullPointerException` w przypadku podania `null` jako argumentu.
- `Serialization` – testy typu "round-trip serialization", czyli serializacja, deserializacja i sprawdzanie różnicy rekordu pierwotnego z rekordem po deserializacji w różnych scenariuszach ewolucji schematu, zapisanego w języku SRL.
- `Compression` – weryfikacja możliwości dodania własnego algorytmu kompresującego do obiektu klasy `SerialCompiler` oraz odpowiednie reakcje obiektu na `null` zwracany przez metdody kompresującą i dekompresującą.

Testy te pozwalają na błyskawiczną weryfikację poprawności kodu w przypadku małych zmian lub głębszego refactoringu.

7 Zastosowane algorytmy

W związku z dużym rozmiarem projektu, nie sposób jest omówić wszystkich wykorzystanych algorytmów, dlatego skupiono się wyłącznie na tych, mających bezpośredni związek z serializacją i deserializacją struktur / rekordów.

7.1 Format binarny struktury

Strukturę definiuje się za pomocą słowa kluczowego `STRUCT`. Jej format binarny składa się z czterobajtowego nagłówka o stałej wartości `0xFE_53_52_4C`, 3-bajtowego hashu nazwy struktury, a następnie z wypisanych w kolejności deklaracji języka SRL, zserializowanych w formacie *big-endian* zmiennych. Poniżej przedstawiony został przykład, obrazujący układ bajtów w pamięci.

7.1.1 Przykład formatu

```
STRUCT Inna
    LET float ulamek = 0.5

STRUCT Struktura
    LET byte bajt = -1
    LET short arr[3] = 1 2 4
    NEST Inna inna
    LET long dluga = 25
```

Format binarny struktury `Struktura` jest następujący:

Offset	Rozmiar	Pole	Typ	Zawartość	Bajty (hex)
0 B	4 B	Nagłówek	–	–	`FE 53 52 4C`
4 B	3 B	Hash	–	–	`?? ?? ??`
7 B	1 B	`bajt`	byte	-1	`FF`
8 B	2 B	`arr[0]`	short	1	`00 01`
10 B	2 B	`arr[1]`	short	2	`00 02`
12 B	2 B	`arr[2]`	short	4	`00 04`
14 B	4 B	`inna.ulamek`	float	0.5	`3F 00 00 00`
18 B	8 B	`dluga`	long	25	`00 00 00 00 00 00 00 19`

Tabela 2: Układ bajtów struktury `Struktura`

Całkowity rozmiar binarny powyższej struktury wynosi **26 bajtów**. Należy zwrócić uwagę, że format ten nie stosuje nagłówków pól, co utrudnia zachowanie kompatybilności w przypadku zmiany układu zmiennych.

7.2 Algorytm serializacji struktury

Za proces serializacji struktury odpowiada metoda `byte[] serializeStruct(Record record)` z klasy `SerialCompiler.Serializer`.

7.2.1 Opis

Algorytm jest realizowany w następujących krokach:

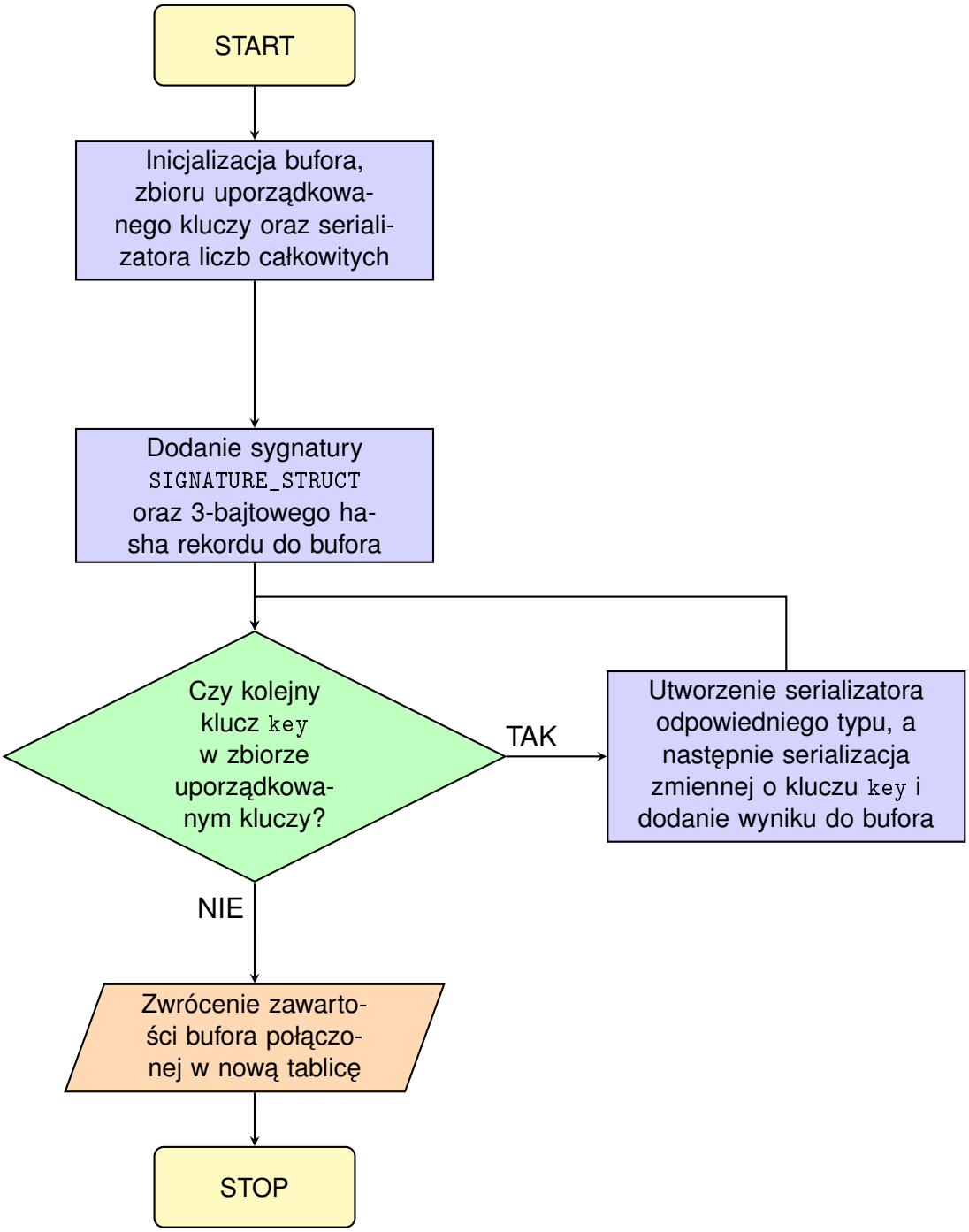
1. Inicjalizacja pustego bufora oraz zbioru uporządkowanego wszystkich kluczy w strukturze oraz obiektu klasy `PrimitiveLike`, odpowiedzialnego za serializację liczb całkowitych typu `int`.
2. Dodanie do bufora zserializowanego nagłówka `SIGNATURE_STRUCT`, obecnego w definicji formatu struktury.
3. Dodanie do bufora 3-bajtowego hasha nazwy rekordu.
4. Dla każdego klucza `key` w zbiorze uporządkowanym kluczy:
   - Utworzenie obiektu klasy `PrimitiveLike`, odpowiedzialnego za serializację prymitywów dla zmiennej o kluczu `key`.
   - Dodanie do bufora zserializowanej zmiennej o kluczu `key`.
5. Zwrócenie zawartości bufora, połączonej w jedną, nową tablicę.

7.2.2 Kod

Poniżej przedstawiono kod w języku Java, odpowiedzialny za serializację struktury.

```java
private byte[] serializeStruct(Record record) {
    ArrayList<byte[]> multiBuffer = new ArrayList<>();
    Set<String> variableList = metadata.getLinkedVariableSet();

    PrimitiveLike<Integer> intInjector = new PrimitiveLike<>(
        Integer.class);

    multiBuffer.add(intInjector.serialize(SIGNATURE_STRUCT));
    multiBuffer.add(Arrays.copyOfRange(
        intInjector.serialize(Utils.fastHash3B(record.RECORD_TYPE)),
        1, 4));

    for (String varName : variableList) {
        VariableInfo varInfo = metadata.getVariableInfo(varName);
        PrimitiveLike<?> prim = new PrimitiveLike<>(varInfo.clazz);

        multiBuffer.add(
            prim.serializeObject(record.getValue(varName))
        );
    }

    return Utils.megaConcat(multiBuffer);
}
```

7.2.3 Schemat blokowy



Rysunek 1: Schemat algorytmu serializacji struktury.

7.3 Algorytm deserializacji struktury

Za proces deserializacji struktury odpowiada metoda `Record deserializeStruct(byte[] bytes, RecordInfo metadata)` z klasy `SerialCompiler.Serializer`. Metoda ta nie zawiera sprawdzania poprawności nagłówka, ani nie wyszukuje odpowiedniego obiektu klasy `RecordInfo` na podstawie hasha nazwy rekordu, gdyż dane do niej przekazywane zostały już pod tym kątem przeanalizowane.

7.3.1 Opis

Algorytm jest realizowany w następujących krokach:

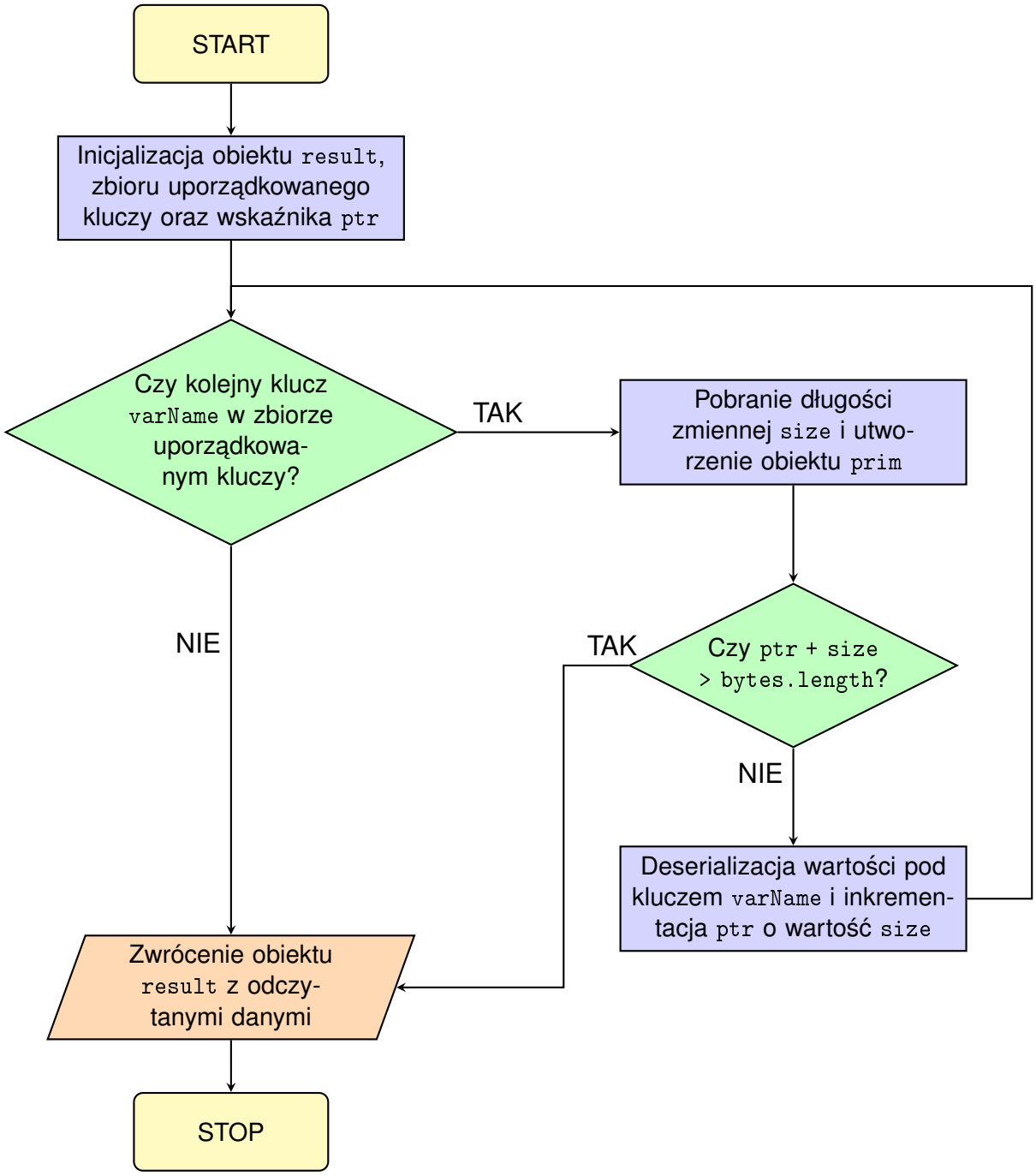
1. Utworzenie nowego obiektu klasy `Record` o nazwie `result` deserializowanego typu, wypełnionego wartościami domyślnymi zmiennych.
2. Pobranie zbioru uporządkowanego kluczy wszystkich zmiennych struktury ze wskazanego obiektu informacji o rekordzie `metadata`.
3. Inicjalizacja wskaźnika przesunięcia `ptr` o rozmiarze 64-bitowym (`long`) i ustawienie go na pozycję startową danych, zaraz za nagłówkiem i hashem nazwy rekordu.
4. Dla każdego klucza `varName` znajdującego się w zbiorze uporządkowanym kluczy:
   - Pobranie szczegółowych informacji o zmiennej o kluczu `key`, w tym jej typu danych, a następnie utworzenie na jej podstawie obiektu klasy `PrimitiveLike` o nazwie `prim`.
   - Wyznaczenie oczekiwanej długości pola w bajtach za pomocą metody `byteLength()` obiektu `prim`.
   - Sprawdzenie, czy suma bieżącej wartości wskaźnika `ptr` oraz rozmiaru pola `size` nie przekracza całkowitej długości wejściowej tablicy bajtów. W przypadku wykrycia próby odczytu poza zakresem tablicy, natychmiastowe przerwanie pętli.
   - Deserializacja wartości z tablicy `bytes` od pozycji wskazanej przez `ptr` za pomocą obiektu `prim` i przypisanie wyniku do obiektu `result` pod kluczem `varName`.
   - Inkrementacja wskaźnika `ptr` o wartość `size`, aby przygotować go do odczytu kolejnego pola.
5. Zwrócenie obiektu `result` zawierającego odczytane dane.

7.3.2 Kod

Poniżej przedstawiono kod w języku Java, odpowiedzialny za deserializację struktury.

```
private Record deserializeStruct(byte[] bytes, RecordInfo metadata) {
    Record result = compiler.makeRecord(metadata.RECORD_TYPE);
    Set<String> variableList = metadata.getLinkedVariableSet();

    long ptr = HEADER_SIZE;
    for (String varName : variableList) {
        VariableInfo varInfo = metadata.getVariableInfo(varName);
        PrimitiveLike<?> prim = new PrimitiveLike<>(varInfo.clazz);

        int size = prim.byteLength();
        if (ptr + size > bytes.length) break;

        result.setValue(varName, prim.deserialize(bytes, (int) ptr));
        ptr += size;
    }

    return result;
}
```

7.3.3 Schemat blokowy



Rysunek 2: Schemat algorytmu deserializacji struktury.

7.4 Format binarny rekordu

Rekord definiuje się za pomocą słowa kluczowego `RECORD`. Jego format binarny składa się z czterobajtowego nagłówka o stałej wartości `0xFF_53_52_4C`, 3-bajtowego hashu nazwy rekordu, a następnie zserializowanych, zgrupowanych w klucze znormalizowane (z zastąpieniem indeksów znakami $*$) grup zmiennych o identycznej ścieżce, ale różnych indeksach. W każdym przypadku, dane są zapisane w postaci *big-endian*.

7.4.1 Format binarny grupy zmiennych

Grupa zmiennych o identycznych kluczach znormalizowanych jest reprezentowana w formacie binarnym jako zwarty blok danych. Składa się on z nagłówka metadanych, definicji wymiarów oraz liniowego ciągu zserializowanych wartości (według spłaszczenia tablicy wielowymiarowej, przy czym wielkości znajdujące się bardziej na lewo w ścieżce są iterowane częściej). Układ bajtów prezentuje się następująco:

Offset	**Rozmiar**	**Zawartość**	**Opis**
0 B	1 B	Rozmiar grupy	Skompresowana stratnie (powiększona lub równa) za pomocą klasy `SizeReducer` wielkość całego obszaru grupy (w bajtach).
1 B	3 B	Hash ścieżki	3-bajtowy hash znormalizowanego klucza.
4 B	$n \times 4$ B	Wymiary	Rozmiary kolejnych wymiarów $L_1, \ldots, L_n$ zapisane jako liczby `int32`.
$4 + (n \times 4)$ B	m B	Dane	Zserializowane wartości składowe, ułożone jedna po drugiej.
$4 + (n \times 4) + m$ B	–	Padding	Padding, wynikający ze stratności kompresji rozmiaru grupy.

Tabela 3: Struktura binarna grupy zmiennych dla n wymiarów.

Powyższe rozumowanie można również odnieść do zwykłych pól bez indeksów. W takim przypadku przyjmuje się $n = 0$, co oznacza całkowite pominięcie sekcji wymiarów w strumieniu danych oraz umieszczenie w sekcji danych dokładnie jednej wartości.

7.4.2 Przykład formatu

```
STRUCT Inna
    LET float ulamek[2] = 0.5 0.5

RECORD Przyklad
    LET byte status = 1
    NEST Inna inna[2]
```

Format binarny rekordu `Przyklad` jest następujący:

Rel. Offset	Rozmiar	Pole	Typ	Zawartość	Bajty (hex)
Nagłówek rekordu					
0 B	4 B	Nagłówek rekordu	–	–	`FF 53 52 4C`
4 B	3 B	Hash rekordu	–	–	`?? ?? ??`
Grupa zmiennej status					
0 B	1 B	Rozmiar grupy	–	≥ 5	`??`
1 B	3 B	Hash ścieżki	–	–	`?? ?? ??`
4 B	1 B	`status`	byte	1	`01`
5 B	p_1 B	Padding	–	–	`00 ...`
Grupa pola inna[*].ulamek[*]					
0 B	1 B	Rozmiar grupy	–	≥ 28	`??`
1 B	3 B	Hash ścieżki	–	–	`?? ?? ??`
4 B	4 B	Wymiar L_1	int32	2	`00 00 00 02`
8 B	4 B	Wymiar L_2	int32	2	`00 00 00 02`
12 B	4 B	`inna[0].ulamek[0]`	float	0.5	`3F 00 00 00`
16 B	4 B	`inna[1].ulamek[0]`	float	0.5	`3F 00 00 00`
20 B	4 B	`inna[0].ulamek[1]`	float	0.5	`3F 00 00 00`
24 B	4 B	`inna[1].ulamek[1]`	float	0.5	`3F 00 00 00`
28 B	p_2 B	Padding	–	–	`00 ...`

Tabela 4: Struktura bajtowa rekordu `Przyklad`.

Całkowity rozmiar binarny powyższego rekordu wynosi co najmniej **40 bajtów**. W praktyce, realny rozmiar może być nieznacznie większy ze względu na padding, ale zawsze stały dla konkretnego typu rekordu. Jako, że padding występuje wyłącznie w przypadku dużych grup (np. tablic), to w przypadku pojedynczych zmiennych wyniesie on 0. Uwzględnienie nagłówków grup w rekordach pozwala na elastyczne modyfikacje układu pól.

7.5 O algorytmach serializacji / deserializacji rekordów

W związku z dużym rozmiarem algorytmów serializacji i deserializacji rekordu, przedstawienie ich opisu, kodu oraz schematu blokowego w sprawozdaniu zajęłoby niezwykle dużo miejsca, a sam proces byłby bardzo pracochłonny. W związku z tym, zachęca się do samodzielnej analizy tych algorytmów w kodzie biblioteki. Znajdują się one w klasie `SerialCompiler.Serializer` w metodach o sygnaturach `byte[] serializeRecord(Record record)` oraz `Record deserializeRecord(byte[] bytes, RecordInfo metadata)`, a ich specyfikacja jest analogiczna do odpowiadających im metod, związanych ze strukturami.

8 Podsumowanie i wnioski

Realizacja projektu pozwoliła na praktyczne zgłębienie i wykorzystanie następujących technologii oraz zagadnień:

- Zaawansowane programowanie obiektowe w języku Java.
- Efektywna praca z ekosystemem IntelliJ IDEA.
- Projektowanie i implementacja testów jednostkowych (metodyka i framework JUnit5).
- Obsługa binarnych formatów danych, w tym zarządzanie kolejnością bajtów (endianowość).
- Implementacja parserów oraz definiowanie własnych struktur danych.

Kluczowym elementem procesu wytwórczego było wdrożenie testów jednostkowych. Pozwoliły one na wczesne wykrycie błędów logicznych i zapewnienie stabilności biblioteki, co przy weryfikacji manualnej byłoby nieefektywne.

W kontekście ograniczeń projektu należy wskazać specyfikę środowiska Java. Brak bezpośredniego dostępu do niskopoziomowych mechanizmów zarządzania pamięcią (dostępnych w językach C czy Rust) wiąże się z pewnym narzutem wydajnościowym oraz większą alokacją pamięci przez maszynę wirtualną.

Obecna wersja biblioteki ma charakter w pełni funkcjonalnego prototypu. Potencjalne wdrożenie produkcyjne powinno zostać poprzedzone testami obciążeniowymi oraz refaktoryzacją dostosowującą kod w celu uzyskania większej czytelności i wydajności.

Mimo tych ograniczeń, rozwiązanie znajduje zastosowanie w systemach wymagających wysokiej integralności danych (np. silniki bazodanowe) oraz w transmisji sieciowej, gdzie priorytetem jest kompresja i integralność pakietów. Zaimplementowany mechanizm `ENSURE` umożliwia skuteczną walidację i rekonstrukcję uszkodzonych danych, co jest krytyczne przy przetwarzaniu informacji z niezaufanych źródeł.